გაკვეთილი 9 Mod 7 (7.3)

გაკვეთილი 1 — კოდის კონფლიქტები და მათი წარმოშობის მიზეზები

1. თეორიული ნაწილი — რა არის კოდის კონფლიქტი?

1.1. რა არის Merge Conflict?

წარმოიდგინეთ, რომ ორი პროგრამისტი მუშაობს ერთსა და იმავე ვებგვერდზე.

ორივეს აქვს ერთი პროექტის ასლი და ორივე ცდილობს ვებგვერდის გაუმჯობესებას.

მაგალითად, პროექტში არსებობს `index.html` ფაილი:

```
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is my website.</p>
</body>
</html>
```

პირველმა პროგრამისტმა შეცვალა სათაური:

```
<h1>Welcome to Our School</h1>
```

მეორე პროგრამისტმა იმავე სათაურში ჩაწერა:

```
<h1>Welcome to Our Academy</h1>
```

ორივე პროგრამისტმა შეცვალა ერთი და იგივე სტრიქონი, მაგრამ განსხვავებულად.

როდესაც Git ამ ცვლილებების გაერთიანებას შეეცდება, მას არ ეცოდინება, რომელი ცვლილება უნდა შეინარჩუნოს.

ასეთ სიტუაციას ეწოდება Merge Conflict — კოდის ვერსიების გაერთიანების კონფლიქტი.

განმარტება

Merge Conflict წარმოიშობა მაშინ, როდესაც Git ვერ ახერხებს სხვადასხვა ვერსიაში განხორციელებული ცვლილებების ავტომატურად გაერთიანებას.

მნიშვნელოვანია გვახსოვდეს:

- კონფლიქტი არ ნიშნავს, რომ კოდი აუცილებლად არასწორად არის დაწერილი.
- კონფლიქტი არ ნიშნავს, რომ პროექტი დაზიანებულია.
- კონფლიქტი ნიშნავს, რომ Git-ს სჭირდება პროგრამისტის გადაწყვეტილება.
- კონფლიქტის მოგვარებისას პროგრამისტი განსაზღვრავს, რომელი ცვლილებები უნდა დარჩეს საბოლოო კოდში.

1.2. რატომ არის მნიშვნელოვანი კონფლიქტების ცოდნა?

რეალურ პროექტებზე ხშირად რამდენიმე პროგრამისტი მუშაობს.

მაგალითად, სკოლის ვებგვერდის შექმნაში მონაწილეობს სამი მოსწავლე:

მოსწავლე	დავალება
მოსწავლე 1	მთავარი გვერდის HTML
მოსწავლე 2	ვებგვერდის CSS დიზაინი
მოსწავლე 3	ნავიგაციის მენიუ

ყველა მათგანი მუშაობს საერთო პროექტზე.

პრობლემა შეიძლება წარმოიშვას, როდესაც ორი მოსწავლე ერთსა და იმავე ფაილს ცვლის.

მაგალითად, ორივე მოსწავლე ცვლის `style.css` ფაილში `body` ელემენტის ფონის ფერს.

პირველი მოსწავლე:

```
body {
    background-color: blue;
}
```

მეორე მოსწავლე:

```
body {
    background-color: green;
}
```

თუ ცვლილებები სხვადასხვა Branch-შია შენახული და მათი გაერთიანება ხდება, Git-მა შეიძლება დააფიქსიროს კონფლიქტი.

მნიშვნელოვანი: ერთი ფაილის რედაქტირება ყოველთვის არ იწვევს კონფლიქტს. კონფლიქტი დამოკიდებულია იმაზე, როგორ და სად შეიცვალა ფაილი და შეუძლია თუ არა Git-ს ცვლილებების გაერთიანება.

2. როგორ მუშაობს Git ცვლილებების გაერთიანებისას?

Git არის ვერსიების კონტროლის სისტემა, რომელიც ინახავს პროექტში განხორციელებულ ცვლილებებს.

როდესაც პროგრამისტები სხვადასხვა Branch-ში მუშაობენ, მათ შეუძლიათ დამოუკიდებლად შეცვალონ კოდი.

შემდეგ Git-ის დახმარებით ხდება ცვლილებების გაერთიანება.

ამ პროცესს ეწოდება Merge.

2.1. რა არის Branch?

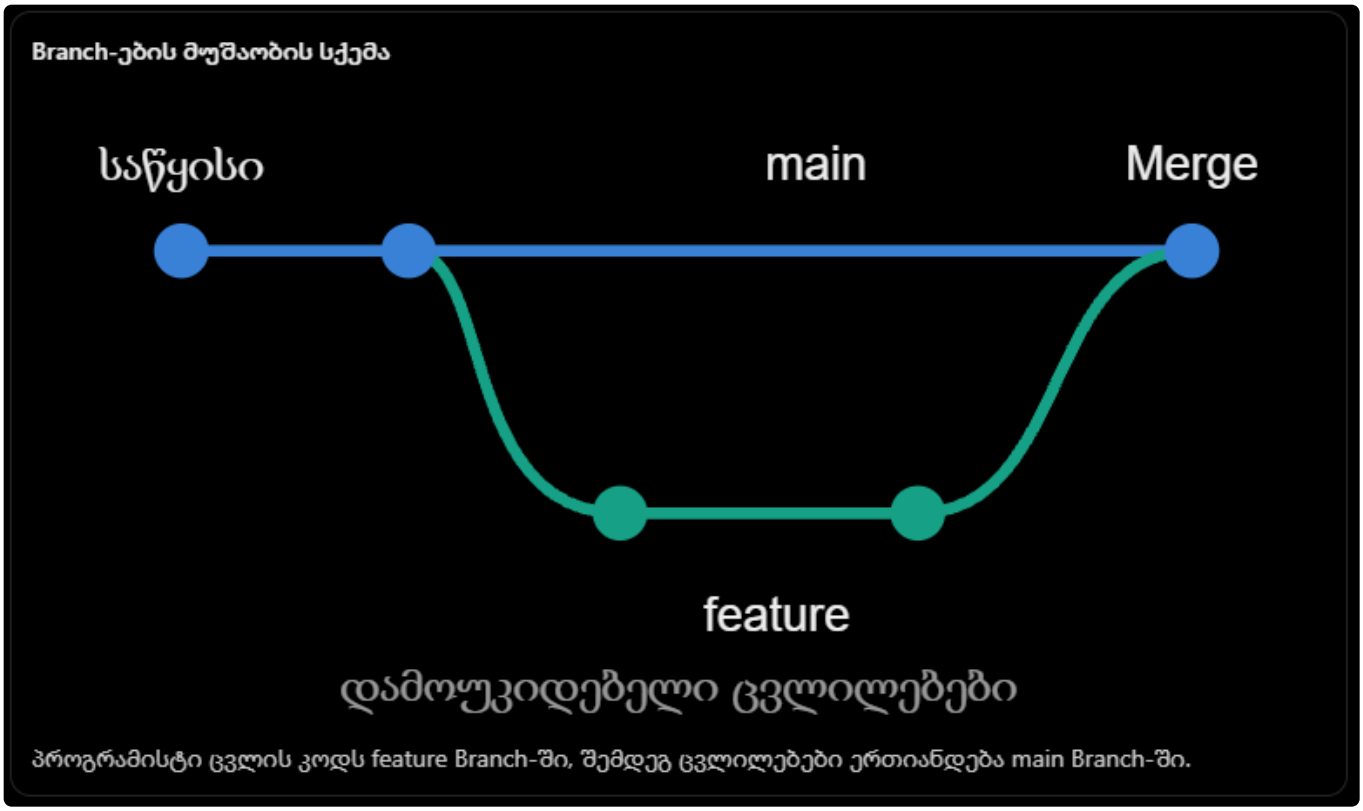
Branch არის პროექტის განვითარების დამოუკიდებელი მიმართულება.

მაგალითად, პროექტში გვაქვს ორი Branch:

- `main` — პროექტის ძირითადი ვერსია.
- `feature` — ახალი ფუნქციონალის ან ცვლილებების შესაქმნელად გამოყოფილი ვერსია.

ორივე Branch-ში შესაძლებელია კოდის შეცვლა.

შემდეგ `feature` Branch-ში განხორციელებული ცვლილებები შეიძლება გაერთიანდეს `main` Branch-ში.



2.2. როდის ხდება ავტომატური გაერთიანება?

განვიხილოთ მაგალითი.

საწყისი კოდი:

```
<h1>My Website</h1>
<p>Hello World</p>
```

`main` Branch-ში შევცვალეთ სათაური:

```
<h1>School Website</h1>
<p>Hello World</p>
```

`feature` Branch-ში შევცვალეთ პარაგრაფი:

```
<h1>My Website</h1>
<p>Welcome Students!</p>
```

რადგან ცვლილებები სხვადასხვა სტრიქონში განხორციელდა და ერთმანეთს არ ეწინააღმდეგება, Git-ს ჩვეულებრივ შეუძლია მათი ავტომატურად გაერთიანება.

საბოლოოდ მივიღებთ:

```
<h1>School Website</h1>
<p>Welcome Students!</p>
```

ამ შემთხვევაში კონფლიქტი არ წარმოიშვა.

2.3. როდის წარმოიშობა კონფლიქტი?

ახლა წარმოვიდგინოთ, რომ ორივე Branch-ში შეიცვალა ერთი და იგივე სათაური.

`main` Branch:

```
<h1>School Website</h1>
```

`feature` Branch:

```
<h1>Student Portal</h1>
```

Git ვერ გადაწყვეტს, რომელი სათაურია სასურველი.

სწორედ ამიტომ გაერთიანებისას წარმოიშობა Merge Conflict.

3. კოდის კონფლიქტების ძირითადი ტიპები

გუნდური მუშაობისას შესაძლებელია რამდენიმე ტიპის კონფლიქტის წარმოშობა.

3.1. ერთი და იმავე სტრიქონის შეცვლა

ეს ერთ-ერთი ყველაზე გავრცელებული შემთხვევაა.

საწყისი კოდი:

```
let score = 10;
```

პირველი პროგრამისტის ცვლილება:

```
let score = 20;
```

მეორე პროგრამისტის ცვლილება:

```
let score = 50;
```

ორივე პროგრამისტმა შეცვალა ერთი და იგივე მნიშვნელობა.

Git-ისთვის გაურკვეველია, რომელი ცვლილება უნდა დარჩეს.

შედეგი: კონფლიქტი.

3.2. ფაილის წაშლა და მისი რედაქტირება

წარმოიდგინეთ, რომ პროექტში გვაქვს ფაილი:

`about.html`

პირველმა პროგრამისტმა გადაწყვიტა, რომ ეს გვერდი აღარ იყო საჭირო და წაშალა.

მეორე პროგრამისტმა კი იმავე ფაილში ახალი ინფორმაცია დაამატა.

Git-ისთვის წარმოიშობა პრობლემა:

- უნდა წაიშალოს ფაილი?
- თუ უნდა შენარჩუნდეს მისი განახლებული ვერსია?

შედეგი: შესაძლებელია Delete/Modify Conflict-ის წარმოშობა.

3.3. ერთი და იმავე ფაილის ახლად შექმნა

მაგალითად, ორ Branch-ში დამოუკიდებლად შეიქმნა ფაილი:

`contact.html`

პირველ Branch-ში:

```
<h1>Contact Us</h1>
```

მეორე Branch-ში:

```
<h1>Get in Touch</h1>
```

ორივე ვერსია ცდილობს ერთი და იმავე სახელის მქონე ფაილის დამატებას.

თუ მათი შიგთავსი განსხვავებულია, Git-მა შეიძლება დააფიქსიროს Add/Add Conflict.

3.4. ერთმანეთთან დაკავშირებული ცვლილებები

კონფლიქტები მხოლოდ ერთი სტრიქონის ცვლილებით არ შემოიფარგლება.

მაგალითად:

პირველმა პროგრამისტმა JavaScript ფუნქციას შეუცვალა სახელი:

```
function calculateTotal() {
    return 100;
}
```

მეორე პროგრამისტმა კი ძველი ფუნქციის სახელი გამოიყენა:

```
let result = calculatePrice();
```

ასეთი ცვლილებების ავტომატური გაერთიანება შეიძლება წარმატებით დასრულდეს, თუმცა პროგრამის მუშაობისას შეცდომა წარმოიშვას.

მნიშვნელოვანი განსხვავება: ეს არის ლოგიკური შეუთავსებლობის მაგალითი და არა აუცილებლად Git-ის Merge Conflict.

ამიტომ პროგრამისტმა გაერთიანების შემდეგ აუცილებლად უნდა შეამოწმოს კოდის გამართულობა.

4. როგორ ამოვიცნოთ Merge Conflict?

როდესაც Git ვერ ახერხებს ცვლილებების ავტომატურად გაერთიანებას, ფაილში შეიძლება გამოჩნდეს სპეციალური ნიშნები.

მაგალითი:

```
<<<<<<< HEAD
<h1>School Website</h1>
=======
<h1>Student Portal</h1>
>>>>>>> feature
```

განვიხილოთ თითოეული ნაწილი.

`<<<<<<< HEAD`

აღნიშნავს პირველი კონფლიქტური ნაწილის დასაწყისს.

ჩვეულებრივი Merge-ის დროს აქ ნაჩვენებია იმ Branch-ის ცვლილება, რომელშიც ამჟამად ვიმყოფებით.

`=======`

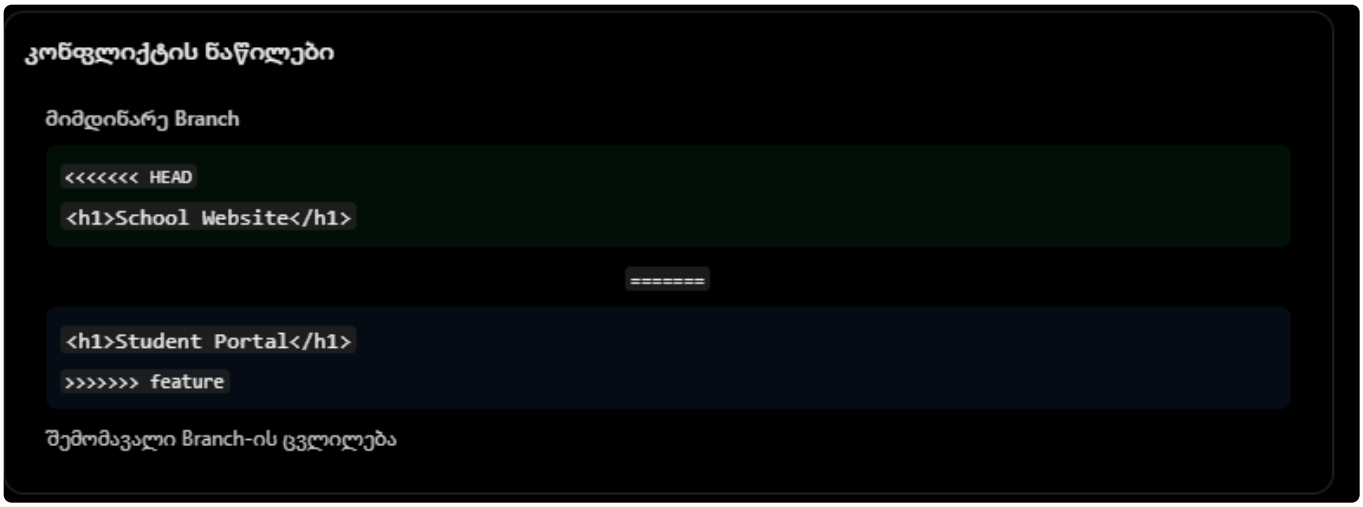
ყოფს ერთმანეთისგან კონფლიქტის ორ ვარიანტს.

`>>>>>>> feature`

აღნიშნავს მეორე ცვლილების დასასრულს.

აქ `feature` არის Branch-ის სახელი, რომლის ცვლილებების გაერთიანებასაც ვცდილობთ.

ვიზუალური მაგალითი



ამ მაგალითში პროგრამისტმა უნდა განსაზღვროს, რომელი სათაური უნდა შენარჩუნდეს.

კონფლიქტის მოგვარების კონკრეტულ მეთოდებს მესამე გაკვეთილში დეტალურად შევისწავლით.

5. პრაქტიკული მაგალითი — კონფლიქტის შექმნა Git-ში

ახლა მოსწავლეები საკუთარი ხელით შექმნიან Merge Conflict-ს.

ამ სავარჯიშოს მიზანია კონფლიქტის აღმოჩენა და არა მისი საბოლოო მოგვარება.

ნაბიჯი 1 — პროექტის შექმნა

VS Code-ში შექმენით ფოლდერი:

`merge-conflict-practice`

გახსენით Terminal და შეასრულეთ:

```
git init
git branch -M main
```

შექმენით `index.html` ფაილი:

```
<!DOCTYPE html>
<html>
<head>
    <title>School Website</title>
```

```
</head>
<body>
    <h1>Welcome</h1>
    <p>Our first Git project.</p>
</body>
</html>
```

შეინახეთ ცვლილებები Git-ში:

```
git add .
git commit -m "Initial project"
```

ნაბიჯი 2 — ახალი Branch-ის შექმნა

Terminal-ში ჩაწერეთ:

```
git checkout -b feature
```

ეს ბრძანება შექმნის ახალ Branch-ს, სახელად `feature`, და გადაგვიყვანს მასში.

ახლა `index.html` ფაილში შეცვალეთ სათაური:

```
<h1>Welcome to Our Academy</h1>
```

შეინახეთ ცვლილებები:

```
git add .
git commit -m "Update heading in feature"
```

ნაბიჯი 3 — main Branch-ში დაბრუნება

Terminal-ში შეასრულეთ:

```
git checkout main
```

ახლა იმავე `index.html` ფაილში შეცვალეთ სათაური:

```
<h1>Welcome to Our School</h1>
```

შეინახეთ:

```
git add .
git commit -m "Update heading in main"
```

ნაბიჯი 4 — Branch-ების გაერთიანება

შეასრულეთ:

```
git merge feature
```

მოსალოდნელია მსგავსი შეტყობინება:

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts
and then commit the result.
```

ეს ნიშნავს, რომ Git-მა ვერ შეძლო ცვლილებების ავტომატურად გაერთიანება.

ნაბიჯი 5 — კონფლიქტის დათვალიერება

გახსენით `index.html`.

ფაილში დაინახავთ დაახლოებით ასეთ ჩანაწერს:

```
<<<<<<< HEAD
<h1>Welcome to Our School</h1>
=======
<h1>Welcome to Our Academy</h1>
>>>>>>> feature
```

მოსწავლეებმა უნდა განსაზღვრონ:

- რომელი ცვლილება ეკუთვნის `main` Branch-ს?
- რომელი ცვლილება ეკუთვნის `feature` Branch-ს?
- რატომ ვერ გააერთიანა Git-მა ცვლილებები?
- რას გვიჩვენებს `=======` ნიშანი?

შემდეგ Terminal-ში შეასრულეთ:

```
git status
```

გამოტანილ ინფორმაციაში გამოჩნდება, რომ `index.html` შეიცავს მოუგვარებელ კონფლიქტს.

ამ ეტაპზე დავაკვირდებით, როგორ გვატყობინებს Git კონფლიქტის არსებობას.

სავარჯიშოს დასრულების შემდეგ, რადგან ჯერ კონფლიქტის მოგვარებას არ ვსწავლობთ, შეგვიძლია მიმდინარე გაერთიანება გავაუქმოთ:

```
git merge --abort
```

ეს ბრძანება დაუმთავრებელ Merge ოპერაციას აუქმებს და ამ მაგალითში გვაბრუნებს გაერთიანებამდე არსებულ მდგომარეობაში.

6. პრაქტიკული სავარჯიშოები მოსწავლეებისთვის

სავარჯიშო 1 — კონფლიქტია თუ არა?

მოსწავლეებმა უნდა განსაზღვრონ, რომელი ცვლილებები გამოიწვევს Merge Conflict-ს.

ყველა მაგალითში ჩათვალეთ, რომ ცვლილებები ორი დამოუკიდებელი Branch-ის სახით განხორციელდა საერთო საწყისი Commit-იდან.

შემთხვევა A

საწყისი კოდი:

```
<h1>Welcome</h1>
<p>Hello</p>
```

პირველმა პროგრამისტმა შეცვალა სათაური:

```
<h1>Our School</h1>
```

მეორემ შეცვალა პარაგრაფი:

```
<p>Hello Students</p>
```

კითხვა: წარმოიშობა თუ არა კონფლიქტი? რატომ?

შემთხვევა B

საწყისი კოდი:

```
h1 {
    color: black;
}
```

პირველმა პროგრამისტმა შეცვალა:

```
h1 {
    color: red;
}
```

მეორემ შეცვალა:

```
h1 {
    color: blue;
}
```

კითხვა: წარმოიშობა თუ არა კონფლიქტი? რატომ?

შემთხვევა C

პროექტში არსებობს `about.html`.

პირველმა პროგრამისტმა ფაილი წაშალა.

მეორემ კი ამავე ფაილში ახალი პარაგრაფი დაამატა.

კითხვა: რა პრობლემა შეექმნება Git-ს?

სავარჯიშო 2 — კონფლიქტის კოდის გაანალიზება

მოცემულია კოდი:

```
<<<<<<< HEAD
let username = "Giorgi";
=======
let username = "Nika";
>>>>>>> feature
```

უპასუხეთ კითხვებს:

1. რომელი ნაწილია მიმდინარე Branch-ის ცვლილება?
2. რომელი ნაწილია შემომავალი Branch-ის ცვლილება?
3. რატომ წარმოიშვა კონფლიქტი?

4. შეუძლია თუ არა Git-ს ავტომატურად გადაწყვიტოს, რომელი სახელი უნდა დარჩეს?

სავარჯიშო 3 — კონფლიქტის პრაქტიკული შექმნა

შექმენით პროექტი სახელად `school-project`.

პროექტში შექმენით `index.html`, რომელსაც ექნება სათაური და პარაგრაფი.

შეასრულეთ შემდეგი მოქმედებები:

1. Git-ის ინიციალიზაცია.
2. პირველი Commit-ის შექმნა.
3. ახალი `design` Branch-ის შექმნა.
4. `design` Branch-ში სათაურის ტექსტის შეცვლა და Commit-ის გაკეთება.
5. `main` Branch-ში დაბრუნება.
6. იმავე სათაურის სხვა ტექსტით შეცვლა და Commit-ის გაკეთება.
7. `git merge design` ბრძანების შესრულება.
8. წარმოშობილი კონფლიქტის აღმოჩენა და მისი მიზეზის ახსნა.

სავარჯიშოს მიზანი: მოსწავლემ დამოუკიდებლად შეძლოს კონფლიქტის შექმნა, აღმოჩენა და მისი მიზეზის განმარტება.

სავარჯიშო 4 — დამატებითი ამოცანა

მოცემულია კონფლიქტური CSS კოდი:

```
<<<<<<< HEAD
.container {
    background-color: yellow;
    padding: 20px;
}
=======
.container {
    background-color: purple;
    padding: 30px;
}
>>>>>>> design
```

განსაზღვრეთ:

1. რომელი თვისებები შეიცვალა ორივე Branch-ში?
2. რომელი მნიშვნელობები ეკუთვნის მიმდინარე Branch-ს?
3. რომელი მნიშვნელობები ეკუთვნის `design` Branch-ს?

4. რა უნდა გადაწყვიტონ პროგრამისტებმა კონფლიქტის მოგვარებამდე?